

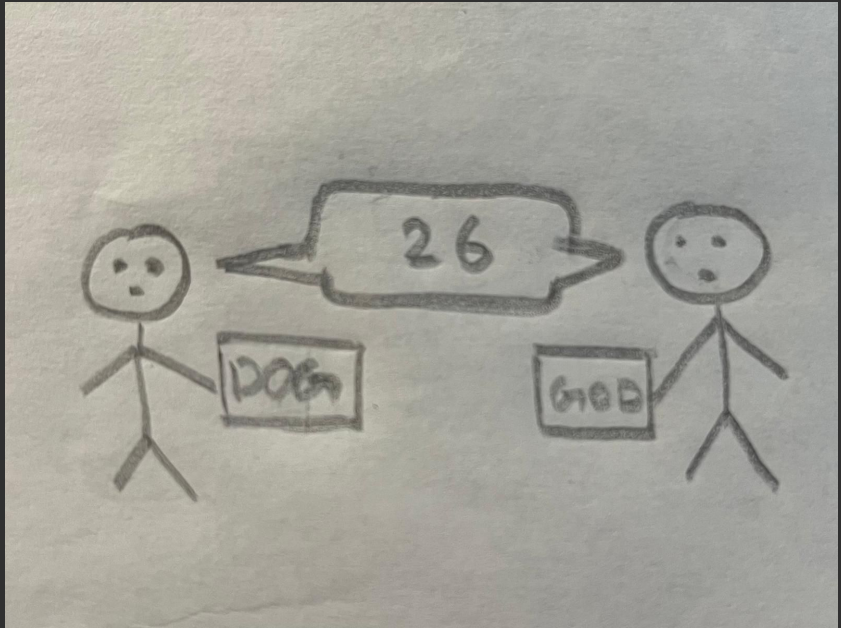
Linear Codes for Error Detection and Correction

CS 880: Final Project Presentation

Ansh Aggarwal, Rohan Chakravarthi, and Yijun Zhu

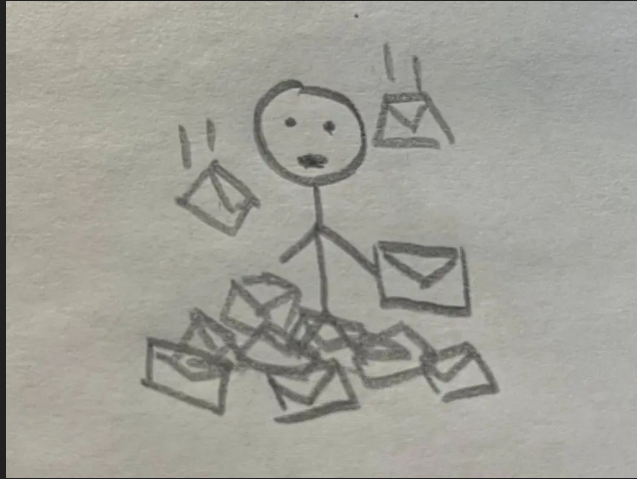
Motivation

Detecting faulty messages over
inconsistent and noisy
communication channels

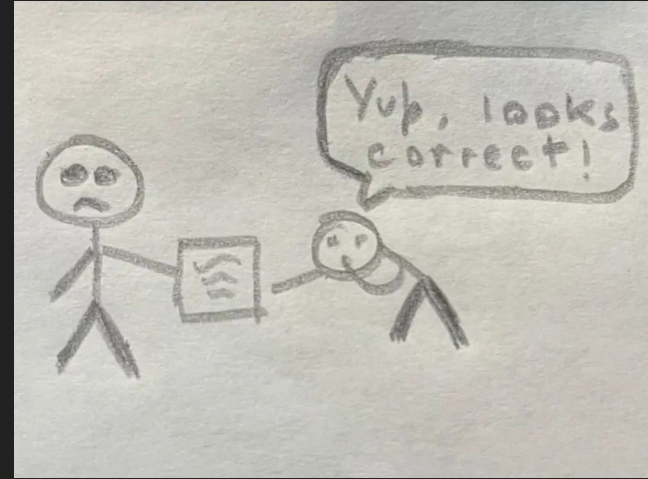


Trivial (but Terrible) Error Correction Ideas

Send the message multiple times and pick the most consistent message



The sender arrives at the receiver's place to verify the message



Goals

- Correctness:
 - Detection: Recognise when the codeword has errors
 - Correction: If the codeword has errors, recover the original message
- Efficiency
 - Time: Polynomial-time
 - Space: Should as close to the message length as possible
- Tolerance:
 - Handle as many errors as possible

Syntax

$$\text{Encode}_s(m) = c$$

$$\text{Decode}_s(c') \in \{\perp, m\}$$

$$\text{Decode}_s(c') = m$$

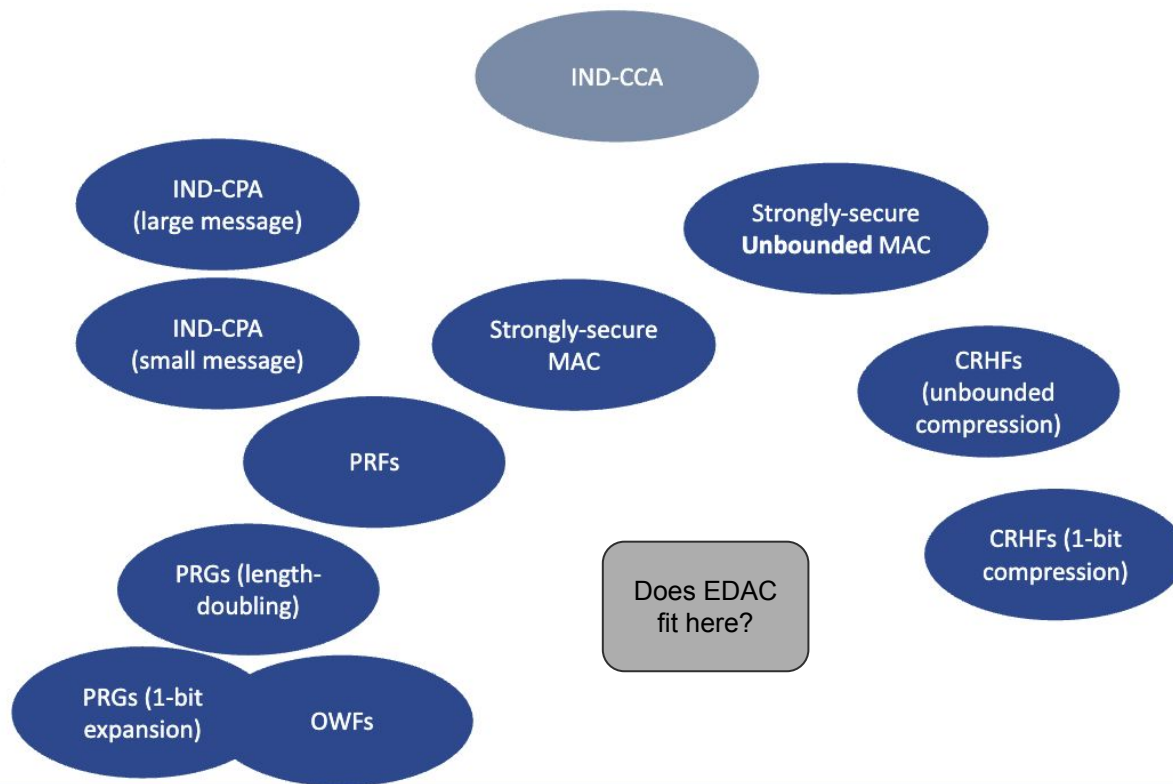
Under both detection and correction, the encode algorithm takes in the message and returns a code word

For detection, the decode algorithm outputs the correct message if there are no errors otherwise, it outputs ABORT with overwhelming probability

For correction, it outputs the original message with overwhelming probability

Revisiting the Crypto Forest

Forest
of
things
seen
so far



Different Notions

- No Adversary: No external attacker is involved; the focus is on error detection and correction.
- No "Game": Unlike IND-CPA, there's no back-and-forth interaction with an adversary.
- Error-Correction Focus: The goal is to handle transmission errors, not defend against attacks.
- Theoretical Model: Assumptions like random noise guide code design without adversarial threats.

Adversarial Channels

- Adversaries don't really exist but the communication channel is considered to be adversarial in that it introduces errors
- When EDAC were first studied (Hamming's 1947 work is usually considered the start of modern EDAC), errors were unbounded because of far less sophisticated equipment
- This isn't strictly true anymore and thus it makes sense to consider less pessimistic adversarial channels

Information Theoretic
Adversarial Channels

VS.

Computationally
Bounded Adversarial
Channels



Basic concepts

- **Error detection rate (R):**
 - Message length / codeword length.
- **Relative error tolerance (ρ):**
 - Max fraction of detectable errors.
- **Error correction:**
 - Max fraction of correctable errors.
- **List-decodable linear code:**
 - Allows recovery of all possible original messages within a certain error radius, outputting a small list of candidates instead of a unique solution.

Constraints of traditional methods

Singleton bound: Provides an upper bound for the size of a code however it implies that for a code with $R > 0$, $\rho < (1 - R)$ and $\rho < 0.5(1 - R)$ for detection and correction respectively

Fraction of correctable errors	Message rate
ρ	R
0	1

Paper 2024

- Detecting and correcting computationally bounded errors:
- Encoding and Transmission.
- Error-Detection: Detecting or recovering
- Error-Correction: The goal is always recover the noisy message.
- Break Singleton bound under computational assumptions.
- Goal: Efficient codes with better rate error tolerance.

No Trusted Setup

- No shared secret keys between sender and receiver
- No public key infrastructure or common reference strings
- Randomness used by encoder is not required by decoder
- Decoding is stateless and public
- Improves applicability in open, distributed systems.
- All security relies on the public randomness.

Two types of codes

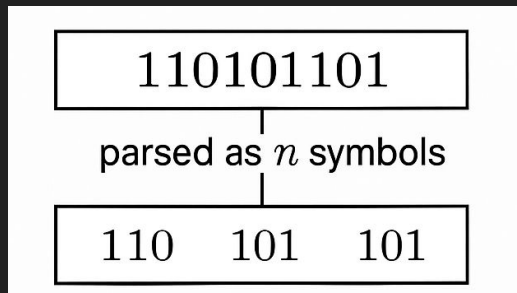
- Randomized Codes:
 - Encoder uses fresh randomness every time
 - Adversary chooses message before randomness
 - Adversary cannot adapt based on randomness
- Self-Seeded Codes:
 - Randomness is fixed and public
 - Adversary (channel) adapts message and errors after seeing the randomness
 - Stronger model, requires stronger assumptions

Differences between the two types of codes

- Randomized Codes:
 - Simpler security model
 - Sufficient to assume one-way functions (OWFs)
 - Message and error are fixed before random coins drawn
- Self-Seeded Codes:
 - More realistic in many cryptographic applications
 - Requires collision-resistant hash functions (CRHFs)
 - Allows adversary to adapt to known randomness
- Self-seeded codes imply both randomized and seeded codes

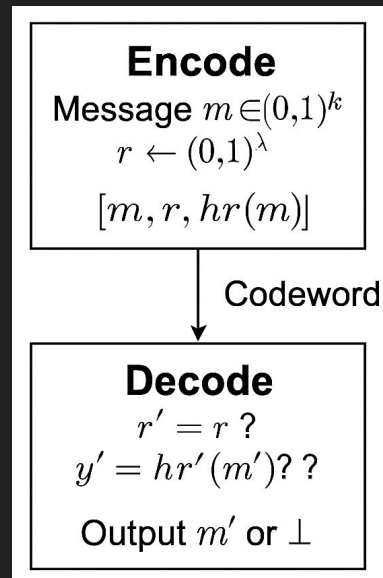
Detection with Sub-Exponential Large Alphabet

- Use collision-resistant hash function H_r with output length λ .
- Encoding:



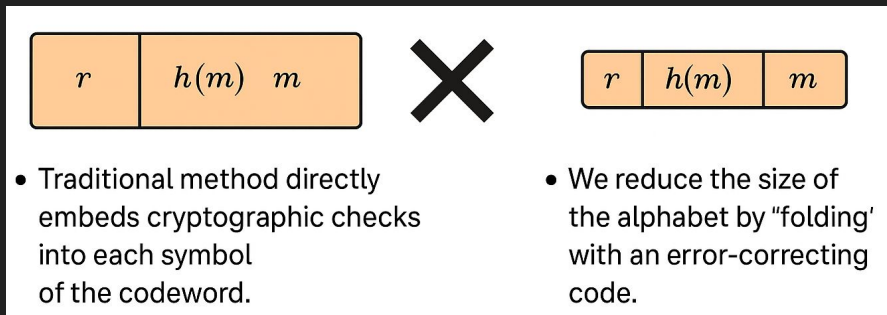
Large alphabet is required to accommodate the message, seed and hash.

Decode:
 y' = hash value of the decoded message.



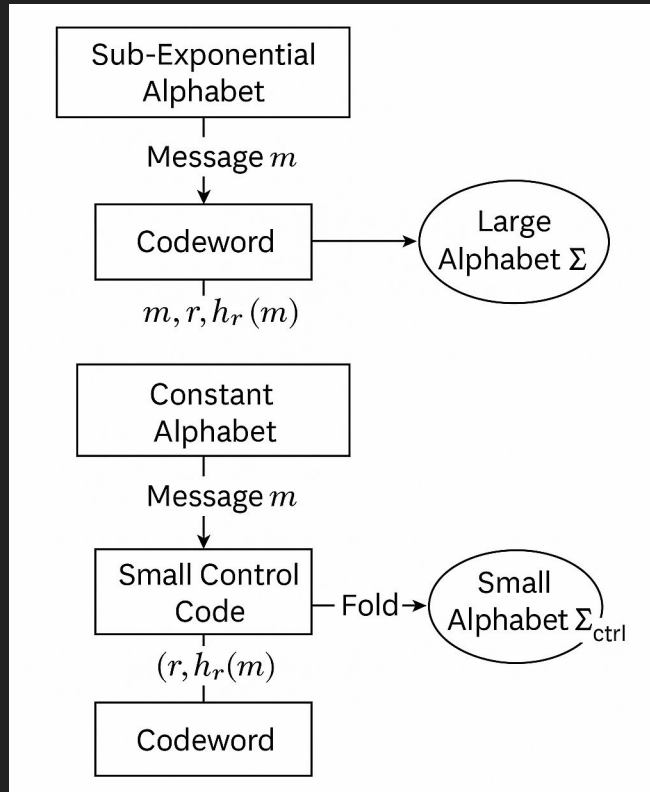
Handling large alphabet is easy, but not realistic

Large alphabets make each symbol carry a lot of information (seed + hash + message), but this requires huge alphabets (sub-exponential size) to fit everything.



Real system only can handle a constant number of alphabets

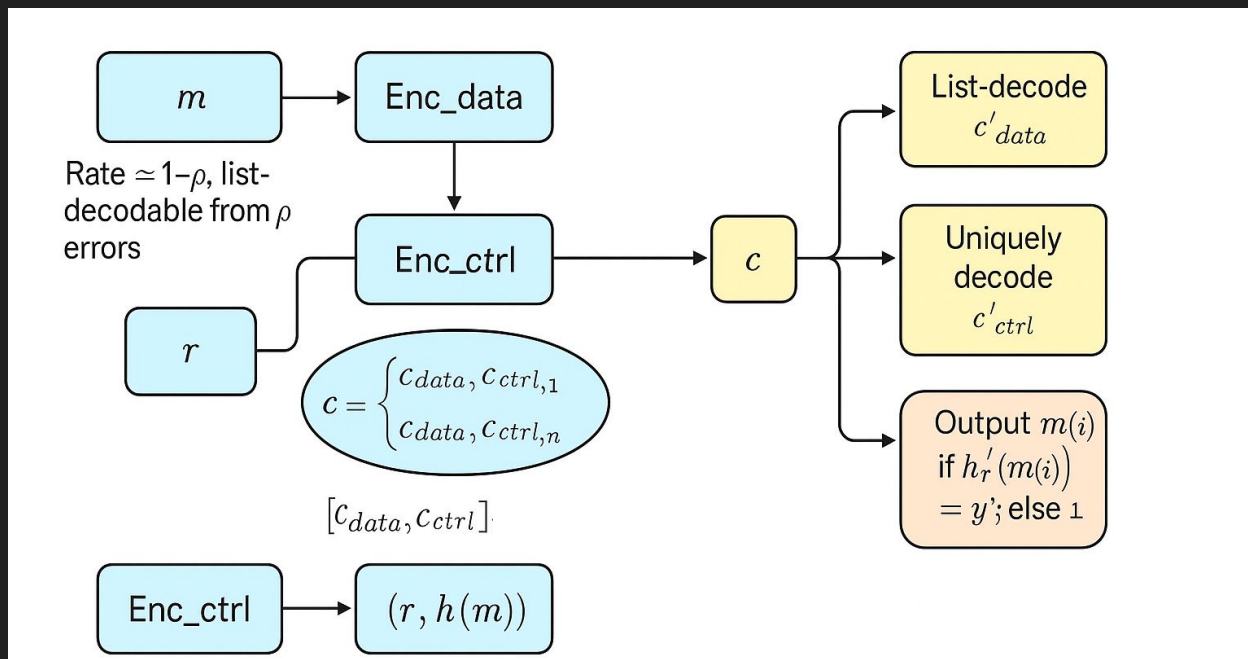
Large Alphabet vs Constant Alphabet



Constant Alphabet Error Correction

- Given the message m with the length k . A random seed r is chosen.
- Use the seeded hash to hash the message $h_r(m)$
- Encode in two ways:
 - $\text{Enc_data}(m)$
 - $\text{Enc_ctrl}(r, h_r(m))$
- Fold, such that for each index position $[i]$
 - $c_i = (\text{Enc_data}(m)[i], \text{Enc_ctrl}(r, h_r(m))[i])$
- Decode: list-decode c'_data and c'_ctrl separately. C'_data gives m_1 to m_l ; c'_ctrl give (r', y') pairs. Placing the values in $h_{r'}(m_i)$ and check whether the combination matches the m_i for same i . If so, the right message is found

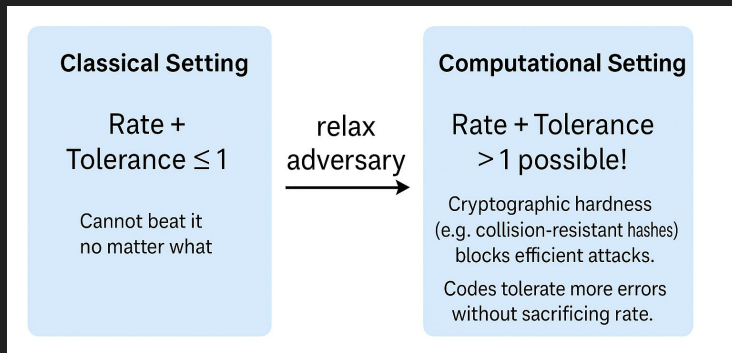
Self-seeded correction



Input = $m, r \rightarrow$ encode $\rightarrow$ fold $\rightarrow$ send $\rightarrow$ list-decode + verify hash $\rightarrow$ recover m .

What Detecting Computationally Bounded Errors is About

- Studies adversarial channels where attackers are limited to efficient (polynomial-time) computation.
- Moves away from classical models that assume attackers can perform any computation.
- Uses cryptographic hardness to allow stronger error correction and detection.
- Designs new codes that work without any trusted setup, only public randomness.



Comparison to Singleton Bound

- Singleton Bound:
 - Detection $\leq 1 - R$
 - Correction $\leq (1-p)/2$
- This work achieves:
 - Detection: $\approx 1/2$
 - Correction: nearly double tolerance over classical bound
- Enabled by computationally bounded error model
- Valid for constant-size alphabets

What's missing?

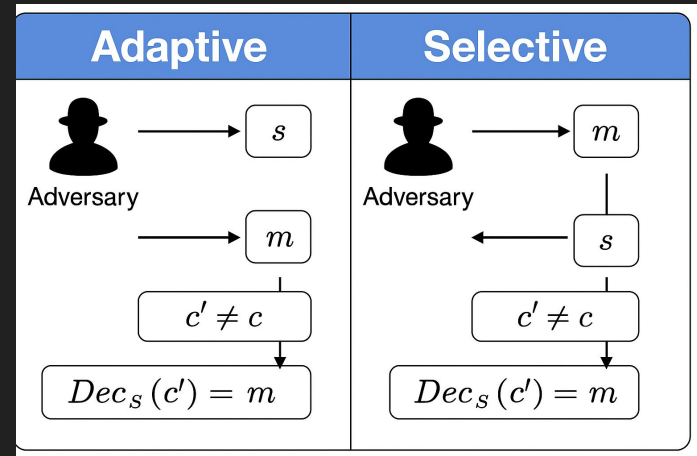
- It achieves strong error detection/correction but only for constant alphabets, not binary.
- Binary codes remained open because folding randomness efficiently into binary was unsolved.
- The follow up paper completes this by constructing binary self-seeded codes under standard cryptographic assumptions.

Goals of the Binary Code Paper

- Construct efficient error-correcting and error-detecting codes
- Protect against arbitrary but efficient adversaries (PPT)
- Avoid any form of trusted setup between encoder and decoder
- Use only foundational assumptions: OWFs or CRHFs
- Surpass traditional bounds with optimal or near-optimal rates

Adaptive vs Selective

- Adaptive: given seed s then the adversary chooses the message m .
- Selective: the adversary chooses the message m first, then a random seed s is chosen.
- The goal is to create a c' such that $c' \neq c$ but $\text{Dec}_s(c') = m$
- Again, adversary is the noisy channel.

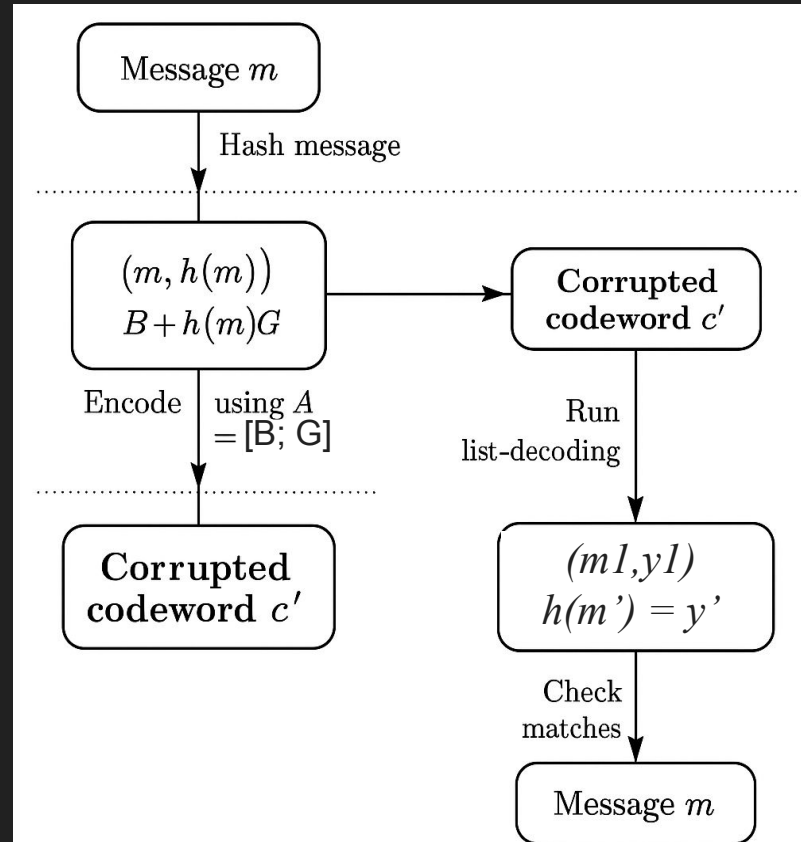


T2CI Hash Details

- T2CI = Targeted 2-input Correlation Intractability.
- Stronger than collision-resistant hash functions.
- Constructed by combining correlation intractable hashing with UOWHF
- Hash circuit has selector bits derived from a universal hash $\hat{h}(m)$
- Where f is a hash family parameterized by selector
- Prevents adversary from correlating m' with same output $h(m')$

Selectively Secure Error Detection, LWE

- Messages m are hashed into $h(m)$ using a public seed.
- The message and its hash are encoded using a list-decodable linear code with generator matrix G .
- Encoding: $c=(m,h(m))*A=mB+h(m)G$, where A is a matrix of $[B; G]$
- Decoding: run list-decoding on c' using the linear code with generator matrix A that gives a list of pairs (m_i', y_i') . For each pair check $h(m') = y'$. Only the true, uncorrupted message pass the hash test.



Selectively Secure Error Correction, LWE

- Same encoding as the previous detection: $c=(m,h(m)) \cdot A$
- Use nested decodable codes.
 - Outer code A with tolerance of p
 - Inner code matrix G with tolerance of $2p$
- The rate is $(k+v)/n \approx$ Blokh-Zyablov bound (k : message length; v : hash length; n : codeword)

LWE-Based Detection Strategy

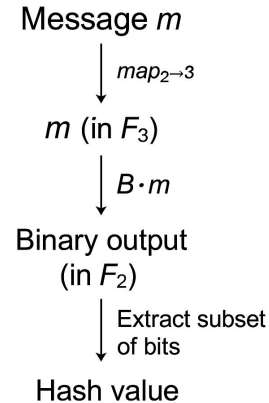
- Use matrix A to encode $(m, h(m))$ via: $c = mB + h(m)G$
- Matrices B, G form public seed s ; $h(m)$ from T2CI hash family
- Codeword = $\text{Enc}_s(m) = (m, h(m)) * A = mB + h(m)G$
- Corruption is modeled as $c' = c + e$
- Decoding: invert A and recover (m', y') ; output m' if $h(m') = y'$, else $\perp$

Crypto Dark Matter Hash Function

- No standard assumption implies hardness.
- Exponentially collision-resistant (conjectured)
- Embeds hardness via mod conversion and masking
- Enables adaptive security without random oracles
- Works well with matrix-based binary codes

Adaptive Hash Innovation

- Crypto Dark Matter: uses mappings between F_2 and F_3
- $\text{map}_{2 \rightarrow 3}(m)$: embeds binary vector into F_3 space
- $\text{map}_{3 \rightarrow 2}$: recovers binary form via secret matrix A
- Final encoding: $\text{Enc}_s(m) = \text{map}_{3 \rightarrow 2}(\text{map}_{2 \rightarrow 3}(m) * B) * A$
- Hash value is a subset of resulting bits

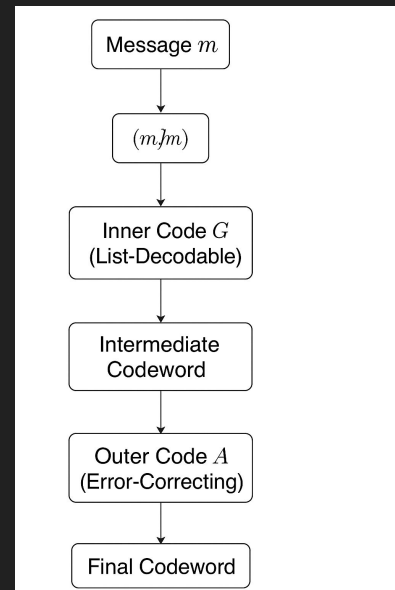


Why Nesting Works

- Inner decoder recovers candidate list of (m', y')
- Outer decoder filters by checking $y' = h(m')$
- Secure under LWE due to T2CI hash unpredictability
- Prevents adversarial codewords from being accepted

Correlation Construction

- Two-level encoding: outer matrix A , inner matrix G
- Inner: list-decodable up to $2p$ errors; Outer: ECC with error detection
- Final codeword: $\text{Enc}_s(m) = (m, h(m))G$ (inner) $\rightarrow$ result encoded with A
- Public matrices provide structure and reproducibility



Adaptively Secure Error Detection, Crypto Dark Matter

- The encoding seed (matrices A and B) is public and known to the adversary.
- The message $m \in \{0,1\}^k$ is mapped to F_3 and multiplied by a random matrix B with dimension $k * v$ over F_3 .
- The result is converted to binary bits and then mapped via matrix A with dimension $2v * n$ over F_2 .

Adaptive Detection Encoding

- $\text{Enc}_s(m) = \text{map}_{3 \rightarrow 2}(\text{map}_{2 \rightarrow 3}(m) * B) * A$
- B, A are public matrices; $\text{map}_{2 \rightarrow 3}$ and $\text{map}_{3 \rightarrow 2}$ are known functions
- Codeword = final binary vector
- Hash = bits extracted from intermediate product

Adaptive Detection Decoding

Input codeword c'



Candidate messages
 m'



Accept m only if $\text{Enc}(m') = c'$

Security: finding $m \neq m'$ such that $\text{Enc}(m) = \text{Enc}(m')$ would break the crypto dark matter assumption.

Adaptive Detection Decoding

- Decoding: list-decode received c' to recover candidate messages m' .
- Accept m' only if $\text{Enc}(m') \approx c'$.
- Security: finding $m \neq m'$ such that $\text{Enc}(m) \approx \text{Enc}(m')$ would break the crypto dark matter assumption.

Adaptively Secure Error Correction, Crypto Dark Matter

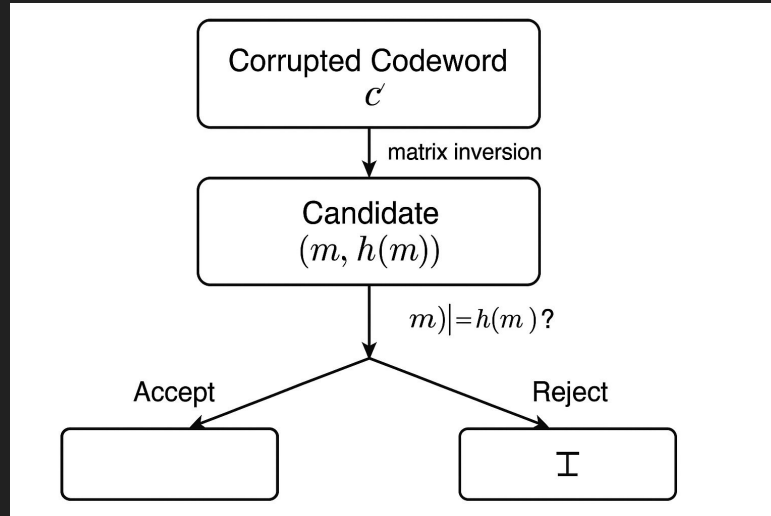
- First, encode message m with an adaptively secure error-detecting code Enc'_s .
- Then, apply a list-decodable outer code Enc_List on top.
- Final encoding: $\text{Encs}(m) = \text{Enc_List}(\text{Enc}'_s(m))$
- Decoding: List-decode outer code to get message candidates.
- Run each through Dec'_s to check for a valid match.
- Works if $p < (p' / 2) * R'$, where p' is the error tolerance of the inner (detection) code, and R' is its rate.

Why This Construction Helps

- Converts message into cryptographic hash via layered transforms
- Hard to forge consistent collision due to field mismatch
- Works even when adversary knows B and A in advance
- Ensures message integrity with high confidence

Decoding and Security

- Decoding = matrix inversion (linear algebra) + T2CI verification
- If hash fails, decoder returns $\perp$ (error detected)
- Security: forging $(m', h(m')) \neq (m, h(m))$ that passes check implies T2CI break
- In turn, T2CI based on LWE; attacker must solve LWE instance to forge

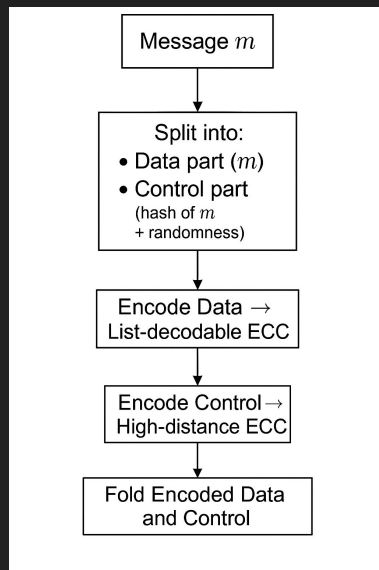


Detection Performance Metrics

- Rate $R \approx 1$ (almost no redundancy added)
- Error detection tolerance: up to $\rho \approx 0.5$ of codeword bits corrupted
- High-rate binary linear code
- No need for cryptographic key or setup beyond public seed
- Practical for high-volume low-latency communication

Construction Overview

- Encoder breaks message into two logical parts:
 - Data: actual content to be transmitted
 - Control: hash of message and encoding randomness
- Uses two separate error-correcting codes:
 - One list-decodable for data
 - One high-distance ECC for control
- Final codeword interleaves both parts ("folded" construction)



Recap: Encoding Process

- Choose random seed r
- Compute hash $h_r(m)$ from a hash family
- Encode message m using ECC: $c_data = \text{Enc_data}(m)$
- Encode control info $(r, h_r(m))$ with ECC: $c_ctrl = \text{Enc_ctrl}(r, h_r(m))$
- Output folded codeword $c_i = (c_data_i, c_ctrl_i)$
- Folding helps ensure both correctness and verifiability

Decoding Process

- Receive corrupted codeword $z = c + e$
- Separate data and control parts: z_data, z_ctrl
- Decode control: get (r', y') from ECC
- List-decode data: get candidate messages $m_1, m_2, \dots, m_l$
- For each m_i , check if $h_{\{r'\}}(m_i) = y'$
- If one match found $\rightarrow$ output m_i ; else $\perp$

Error Correction Recap

- Remember, correction works when $\rho < 1/2$
- Achieves rate $R = 1 - \rho - \epsilon$
- Example: if $\rho = 0.25$, $R \approx 0.75$
- List-decodable ECC identifies candidate messages
- Hash filtering isolates the correct message deterministically
- Reliable decoding with overwhelming probability

Why does it work?

- High-distance control code prevents ambiguity in r , $h_r(m)$
- List-decodable data ECC ensures original m is in the list
- Hash check eliminates incorrect candidates
- Breaking this implies hash collision (CRHF/UOWHF), which is infeasible

Comparison to GHY20 and SW24

- GHY20: needs random oracle, not instantiable
- SW24: large alphabets, not binary
- This work:
 - Binary alphabet
 - Public randomness only
 - LWE-based or conjectured hash assumptions

Differences from 2024 to 2025 Paper Findings

- Alphabet Size
- Security Model Extension
- Assumptions Used
- Error Correction Efficiency
- Modular Constructions

Comparisons Between 2024 and 2025 Papers (Cont.)

- Security Notions
 - 2024: Randomized/self-seeded security.
 - 2025: Fully supports adaptive and selective attacks in public-randomness model.
- Code Structures
 - 2024: Folded codewords with shared randomness.
 - 2025: Encodings based on matrix multiplication and field conversion for flexibility.

2025 model breakdown

- Title: Binary Codes for Error Detection and Correction in a Computationally Bounded World
- Problem: Correcting and detecting errors over adversarial noisy channels
- Setting: Channel introduces arbitrary errors, but is computationally bounded (PPT)
- Goal: Achieve strong error resilience in the binary alphabet using cryptographic hardness

Use Cases

- Secure file storage and retrieval (e.g., cloud, disk)
- Tamper detection in firmware or logs
- Adversarial communication protocols (e.g., sensors, satellites)
- Any application requiring public verifiability and no trusted setup

Open Problems

- Hardness of Crypto Dark Matter
- Adaptive Correction from Standard Assumptions
- Binary Alphabet for Publicly Verifiable Codes
- Efficiency Optimization
- Applications to Streaming or Real-Time Channels